쿼리 증명 시스템 상세 분석 (Query Proof System)

파일 개요

- **파일 경로**: `crates/proof-of-sql/src/sql/proof/query_proof.rs`
- **주요 역할**: SQL 쿼리를 영지식 증명으로 변환하는 핵심 시스템
- **핵심 기능**: 쿼리 실행 결과에 대한 증명 생성 및 검증

모듈 구조 및 의존성

주요 Import 분석

```
use super::{
make_sumcheck_state::make_sumcheck_prover_state, FinalRoundBuilder, FirstRoundBu
ProofPlan, QueryData, QueryResult, SumcheckMleEvaluations, SumcheckRandomScalars
VerificationBuilderImpl,
};
```

설명: 증명 생성을 위한 핵심 빌더 패턴과 sumcheck 프로토콜 관련 컴포넌트들을 가져옵니다.

```
use crate::{
base::{
```

```
bit::BitDistribution,

commitment::{Commitment, CommitmentEvaluationProof, CommittableColumn},

database::{

ColumnRef, CommitmentAccessor, DataAccessor, LiteralValue, MetadataAccessor,

OwnedTable, Table, TableRef,

},

// ... 기타 모듈들

},

proof_primitive::sumcheck::SumcheckProof,

utils::log,

};
```

설명: 데이터베이스 접근, commitment 스킴, sumcheck 증명 등 영지식 증명의 기본 구성 요소들을 가져옵니다.

핵심 데이터 구조

1. FirstRoundMessage 구조체

```
#[derive(Clone, Serialize, Deserialize)]

pub  struct FirstRoundMessage<C> {

/// Length of the range of generators we use

pub range_length: usize,

pub post_result_challenge_count: usize,

/// Chi evaluation lengths
```

```
pub chi_evaluation_lengths: Vec<usize>,

/// Rho evaluation lengths

pub rho_evaluation_lengths: Vec<usize>,

/// First Round Commitments

pub round_commitments: Vec<C>,

}
```

설명:

- **range_length**: 증명에 사용되는 데이터 범위의 길이
- **post_result_challenge_count**: 결과 후 챌린지 개수 (Fiat-Shamir 변환용)
- **chi_evaluation_lengths**: χ(카이) 다항식 평가 길이들
- **rho_evaluation_lengths**: ρ(로) 다항식 평가 길이들
- **round_commitments**: 첫 번째 라운드의 commitment들

2. FinalRoundMessage 구조체

```
#[derive(Clone, Serialize, Deserialize)]

pub  struct FinalRoundMessage<C> {

pub subpolynomial_constraint_count: usize,

/// Final Round Commitments

pub round_commitments: Vec<C>,

/// Bit distributions

pub bit_distributions: Vec<BitDistribution>,
```

```
}
```

설명:

- **subpolynomial_constraint_count**: 부분 다항식 제약 조건의 개수
- **round_commitments**: 최종 라운드의 commitment들
- **bit_distributions**: 비트 분포 정보 (범위 증명용)

3. QueryProof 메인 구조체

```rust
#[derive(Clone, Serialize, Deserialize)]
pub  struct QueryProof<CP: CommitmentEvaluationProof> {
pub(super) first_round_message: FirstRoundMessage<CP::Commitment>,
pub(super) final_round_message: FinalRoundMessage<CP::Commitment>,
/// Sumcheck Proof
pub(super) sumcheck_proof: SumcheckProof<CP::Scalar>,
pub(super) pcs_proof_evaluations: QueryProofPCSProofEvaluations<CP::Scalar>,
/// Inner product proof of the MLEs' evaluations
pub(super) evaluation_proof: CP,
}
```

설명: 완전한 쿼리 증명을 구성하는 모든 요소들을 포함하는 메인 구조체입니다.

핵심 함수 분석

1. get_index_range 함수

```
fn get_index_range<'a>(
accessor: &dyn MetadataAccessor,
table_refs: impl IntoIterator<Item = &'a TableRef>,
) -> (usize, usize) {
table_refs
.into_iter()
.map(|table_ref| {
let length = accessor.get_length(table_ref);
let offset = accessor.get_offset(table_ref);
(offset, offset + length)
})
.reduce(|(min_start, max_end), (start, end)| (min_start.min(start), max_end.max(
// Only applies to `EmptyExec` where there are no tables
.unwrap_or((0, 1))
}
```

설명:

- 쿼리에서 참조하는 모든 테이블의 행 번호 범위를 계산합니다
- 가장 작은 offset과 가장 큰 offset + length를 찾아 전체 인덱스 범위를 결정합니다
- 빈 쿼리의 경우 기본값 (0, 1)을 반환합니다

2. QueryProof::new 함수 - 증명 생성 과정

2.1 초기 설정 및 데이터 준비

```
pub  fn new(
expr: &(impl ProofPlan + Serialize),
accessor: &impl DataAccessor<CP::Scalar>,
setup: &CP::ProverPublicSetup<'_>,
params: &[LiteralValue],
) -> PlaceholderResult<(Self, OwnedTable<CP::Scalar>)> {
log::log_memory_usage("Start");


let (min_row_num, max_row_num) = get_index_range(accessor, &expr.get_table_refer
let initial_range_length = (max_row_num - min_row_num).max(1);
let alloc = Bump::new();
```

설명:

- **expr**: 증명할 쿼리의 실행 계획 (ProofPlan)
- **accessor**: 데이터베이스 접근자
- **setup**: 증명자의 공개 설정
- **params**: 쿼리 매개변수들
- 메모리 사용량을 로깅하고 범위 길이를 계산합니다

2.2 테이블 데이터 수집

```
let total_col_refs = expr.get_column_references();
let table_map: IndexMap<TableRef, Table<CP::Scalar>> = expr
.get_table_references()
```

```
.into_iter()
.map(|table_ref| {
let idents: IndexSet<Ident> = total_col_refs
.iter()
.filter(|col_ref| col_ref.table_ref() == table_ref)
.map(ColumnRef::column_id)
.collect();
(table_ref.clone(), accessor.get_table(&table_ref, &idents))
})
.collect();
```

설명:

- 쿼리에서 참조하는 모든 컬럼을 수집합니다
- 각 테이블별로 필요한 컬럼들만 선별하여 테이블 맵을 구성합니다
- 메모리 효율성을 위해 필요한 컬럼만 로드합니다

2.3 첫 번째 라운드 - 쿼리 실행 및 결과 생성

```
// Prover First Round: Evaluate the query && get the right number of post result
let  mut first_round_builder = FirstRoundBuilder::new(initial_range_length);
let query_result =
expr.first_round_evaluate(&mut first_round_builder, &alloc, &table_map, params)?
let owned_table_result = OwnedTable::from(&query_result);
```

```
let provable_result = query_result.into();
```

설명:

- FirstRoundBuilder를 사용하여 쿼리를 실행합니다
- 쿼리 결과를 생성하고 증명 가능한 형태로 변환합니다
- 결과는 나중에 검증자가 확인할 수 있도록 소유된 테이블로 저장됩니다

2.4 Sumcheck 변수 계산 및 Commitment 생성

```
let range_length = first_round_builder.range_length();
let num_sumcheck_variables = cmp::max(log2_up(range_length), 1);
assert!(num_sumcheck_variables > 0);
let post_result_challenge_count = first_round_builder.num_post_result_challenges

// commit to any intermediate MLEs
let first_round_commitments =
first_round_builder.commit_intermediate_mles(min_row_num, setup);
```

설명:

- **num_sumcheck_variables**: sumcheck에 필요한 변수 개수 ($\log_2$(range_length))
- 중간 다선형 확장(MLE)들에 대한 commitment를 생성합니다
- 이는 나중에 검증자가 증명자의 계산을 확인하는 데 사용됩니다

2.5 Fiat-Shamir 변환을 위한 Transcript 구성

```
// construct a transcript for the proof
let  mut transcript: Keccak256Transcript = Transcript::new();
transcript.extend_as_le([SETUP_HASH]);
transcript.challenge_as_le();
transcript.extend_serialize_as_le(expr);
transcript.challenge_as_le();
transcript.extend_serialize_as_le(&owned_table_result);
transcript.challenge_as_le();
```

설명:

- Keccak256 해시를 사용한 transcript를 생성합니다
- 설정 해시, 쿼리 표현식, 결과를 순차적으로 추가합니다
- 각 단계마다 챌린지를 생성하여 무작위성을 보장합니다

2.6 컬럼 Commitment 추가

```
for commitment in CP::Commitment::compute_commitments(
&expr
.get_column_references()
.into_iter()
.map(|col| {
CommittableColumn::from(accessor.get_column(&col.table_ref(), &col.column_id()))
})
.collect_vec(),
```

```
min_row_num,
setup,
) {
transcript.extend_serialize_as_le(&commitment);
}
transcript.challenge_as_le();
```

설명:

- 쿼리에서 참조하는 모든 컬럼에 대한 commitment를 계산합니다
- 각 commitment를 transcript에 추가하여 검증 가능하게 만듭니다

2.7 최종 라운드 - Sumcheck 준비

```
let post_result_challenges =
core::iter::repeat_with(|| transcript.scalar_challenge_as_be())
.take(post_result_challenge_count)
.collect();

let  mut final_round_builder =
FinalRoundBuilder::new(num_sumcheck_variables, post_result_challenges);

expr.final_round_evaluate(&mut final_round_builder, &alloc, &table_map, params)?
```

설명:

- 결과 후 챌린지들을 생성합니다 (Fiat-Shamir 변환)
- FinalRoundBuilder로 최종 라운드 평가를 수행합니다
- 이 단계에서 sumcheck에 필요한 다항식들이 구성됩니다

2.8 Sumcheck 증명 생성

```
let state = make_sumcheck_prover_state(
final_round_builder.sumcheck_subpolynomials(),
num_sumcheck_variables,
&SumcheckRandomScalars::new(&random_scalars, range_length, num_sumcheck_variable
);
transcript.challenge_as_le();


// create the sumcheck proof -- this is the main part of proving a query
let span = span!(Level::DEBUG, "Sumcheck with initialization").entered();
let  mut evaluation_point = vec![Zero::zero(); state.num_vars];
let sumcheck_proof = SumcheckProof::create(&mut transcript, &mut evaluation_poin
span.exit();
```

설명:

- **핵심 부분**: 이것이 쿼리 증명의 메인 파트입니다
- sumcheck 증명자 상태를 만들고 실제 sumcheck 증명을 생성합니다
- evaluation_point는 sumcheck 프로토콜의 결과로 얻어지는 평가점입니다

2.9 MLE 평가 및 Inner Product 증명

```
// evaluate the MLEs used in sumcheck except for the result columns

let span = span!(Level::DEBUG, "initialize evaluation_vec").entered();

let  mut evaluation_vec = vec![Zero::zero(); range_length];

span.exit();

compute_evaluation_vector(&mut evaluation_vec, &evaluation_point);
```

설명:

- sumcheck에서 얻은 평가점을 사용하여 평가 벡터를 계산합니다
- 이 벡터는 모든 MLE들을 평가하는 데 사용됩니다

```
// fold together the pre result MLEs -- this will form the input to an inner pro

// of their evaluations (fold in this context means create a random linear combi

let random_scalars: Vec<_> =

core::iter::repeat_with(|| transcript.scalar_challenge_as_be())

.take(

pcs_proof_evaluations.first_round.len()

+ pcs_proof_evaluations.column_ref.len()

+ pcs_proof_evaluations.final_round.len(),

)

.collect();
```

설명:

- "Fold"는 여러 MLE들을 무작위 선형 결합으로 합치는 과정입니다

- 이를 통해 여러 개의 증명을 하나로 배치(batch)할 수 있습니다

```
// finally, form the inner product proof of the MLEs' evaluations

let evaluation_proof = CP::new(

&mut transcript,

&folded_mle,

&evaluation_point,

min_row_num as u64,

setup,

);
```

설명:

- 최종적으로 MLE 평가들에 대한 inner product 증명을 생성합니다
- 이는 commitment된 데이터와 실제 평가값이 일치함을 증명합니다

증명 생성 플로우 요약

1. **데이터 준비**: 쿼리 참조 테이블들의 범위 계산 및 데이터 로드
2. **첫 번째 라운드**: 쿼리 실행 및 중간 결과 commitment
3. **Transcript 구성**: Fiat-Shamir 변환을 위한 해시 체인 구축
4. **최종 라운드**: Sumcheck용 다항식 구성
5. **Sumcheck 증명**: 메인 증명 생성 (SQL → 다항식 변환의 핵심)
6. **MLE 평가**: 평가점에서의 다선형 확장 계산
7. **Inner Product 증명**: Commitment 일관성 증명

다음 단계

이 분석을 통해 SQL 쿼리가 어떻게 다항식 형태로 변환되고, sumcheck 프로토콜을 통해 증명되는지 이해할 수 있습니다. 다음으로는 Filter 실행 계획과 쿼리 파싱 시스템►► 분석하여 전체 SQL 처리 파이프라인을 완성하겠습니다.

검증 과정 분석 (QueryProof::verify)

1. 검증 초기화 및 유효성 검사

```
pub  fn verify(

self,

expr: &(impl ProofPlan + Serialize),

accessor: &impl CommitmentAccessor<CP::Commitment>,

result: OwnedTable<CP::Scalar>,

setup: &CP::VerifierPublicSetup<'_>,

params: &[LiteralValue],

) -> QueryResult<CP::Scalar> {

log::log_memory_usage("Start");



let table_refs = expr.get_table_references();

let (min_row_num, _) = get_index_range(accessor, &table_refs);

let num_sumcheck_variables = cmp::max(log2_up(self.first_round_message.range_len
```

```
assert!(num_sumcheck_variables > 0);
```

설명:

- 검증자는 증명자와 동일한 방식으로 기본 매개변수들을 계산합니다
- 테이블 참조와 sumcheck 변수 개수를 재계산하여 일관성을 확인합니다

2. 비트 분포 유효성 검사

```
// validate bit decompositions
for dist in &self.final_round_message.bit_distributions {
if !dist.is_valid() {
Err(ProofError::VerificationError {
error: "invalid bit distributions",
})?;
} else  if !dist.is_within_acceptable_range() {
Err(ProofError::VerificationError {
error: "bit distribution outside of acceptable range",
})?;
}
}
```

설명:

- 범위 증명에 사용되는 비트 분포가 유효한지 검사합니다
- 허용 가능한 범위 내에 있는지 확인하여 악의적인 증명을 방지합니다

3. Transcript 재구성

```
// construct a transcript for the proof
let  mut transcript: Keccak256Transcript = Transcript::new();
transcript.extend_as_le([SETUP_HASH]);
transcript.challenge_as_le();
transcript.extend_serialize_as_le(expr);
transcript.challenge_as_le();
transcript.extend_serialize_as_le(&result);
transcript.challenge_as_le();
```

설명:

- 검증자는 증명자와 정확히 동일한 순서로 transcript를 재구성합니다
- 이를 통해 Fiat-Shamir 변환의 챌린지들을 동일하게 생성할 수 있습니다

4. Commitment 검증

```
for commitment in expr
.get_column_references()
.into_iter()
.map(|col| accessor.get_commitment(&col.table_ref(), &col.column_id()))
{
transcript.extend_serialize_as_le(&commitment);
}
```

```
transcript.challenge_as_le();
```

설명:

- 검증자는 CommitmentAccessor를 통해 미리 계산된 commitment들을 가져옵니다
- 이 commitment들은 데이터베이스 소유자가 미리 계산하여 공개한 것입니다

5. Sumcheck 검증

```
// verify sumcheck up to the evaluation check
let subclaim = self.sumcheck_proof.verify_without_evaluation(
&mut transcript,
num_sumcheck_variables,
&Zero::zero(),
)?;
```

설명:

- **핵심 검증 단계**: sumcheck 프로토콜을 검증합니다
- `verify_without_evaluation` 은 평가 검사를 제외한 sumcheck 검증을 수행합니다
- 결과로 subclaim을 얻는데, 이는 평가점과 기대값을 포함합니다

6. MLE 평가 검증

```
let sumcheck_evaluations = SumcheckMleEvaluations::new(
self.first_round_message.range_length,
chi_evaluation_lengths,
```

```
self.first_round_message.rho_evaluation_lengths.clone(),
&subclaim.evaluation_point,
&sumcheck_random_scalars,
&self.pcs_proof_evaluations.first_round,
&self.pcs_proof_evaluations.final_round,
);
```

설명:

- sumcheck에서 얻은 평가점을 사용하여 MLE 평가들을 계산합니다
- χ(chi)와 ρ(rho) 평가 길이들을 사용하여 정확한 평가를 수행합니다

7. 검증자 평가 수행

```
let verifier_evaluations = expr.verifier_evaluate(
&mut builder,
&evaluation_accessor,
Some(&result),
&chi_eval_map,
params,
)?;
// compute the evaluation of the result MLEs
let result_evaluations = result.mle_evaluations(&subclaim.evaluation_point);
// check the evaluation of the result MLEs
if verifier_evaluations.column_evals() != result_evaluations {
```

```
Err(ProofError::VerificationError {
error: "result evaluation check failed",
})?;
}
```

설명:

- 검증자는 쿼리 표현식을 평가하여 기대되는 결과를 계산합니다
- 실제 결과의 MLE 평가와 비교하여 일치하는지 확인합니다
- **중요**: 이 단계에서 쿼리 결과의 정확성이 검증됩니다

8. Sumcheck 평가 검사

```
// perform the evaluation check of the sumcheck polynomial
if builder.sumcheck_evaluation() != subclaim.expected_evaluation {
Err(ProofError::VerificationError {
error: "sumcheck evaluation check failed",
})?;
}
```

설명:

- sumcheck 다항식의 평가값이 기대값과 일치하는지 최종 확인합니다
- 이는 sumcheck 프로토콜의 완전성을 보장하는 마지막 단계입니다

9. Inner Product 증명 검증

```
// finally, check the MLE evaluations with the inner product proof
self.evaluation_proof
.verify_batched_proof(
&mut transcript,
&pcs_proof_commitments,
&evaluation_random_scalars,
&pcs_proof_evaluations,
&subclaim.evaluation_point,
min_row_num as u64,
self.first_round_message.range_length,
setup,
)
.map_err(|_e| ProofError::VerificationError {
error: "Inner product proof of MLE evaluations failed",
})?;
```

설명:

- 최종적으로 MLE 평가들에 대한 inner product 증명을 검증합니다
- 이는 commitment된 데이터와 실제 평가값의 일관성을 확인합니다
- 배치 검증을 통해 여러 증명을 효율적으로 처리합니다

10. 검증 완료 및 결과 반환

```
let verification_hash = transcript.challenge_as_le();
```

```
log::log_memory_usage("End");


Ok(QueryData {
table: result,
verification_hash,
})
```

설명:

- 최종 검증 해시를 생성하여 전체 증명 과정의 무결성을 보장합니다
- 검증된 결과 테이블과 검증 해시를 포함한 QueryData를 반환합니다

검증 플로우 요약

1. **초기화**: 기본 매개변수 재계산 및 비트 분포 유효성 검사
2. **Transcript 재구성**: 증명자와 동일한 순서로 해시 체인 구축
3. **Commitment 확인**: 미리 공개된 데이터 commitment들 검증
4. **Sumcheck 검증**: 핵심 다항식 증명 프로토콜 검증
5. **MLE 평가**: 평가점에서의 다선형 확장 계산 및 비교
6. **결과 검증**: 쿼리 결과의 정확성 확인
7. **Inner Product 검증**: Commitment 일관성 최종 확인
8. **완료**: 검증 해시 생성 및 결과 반환

SQL에서 다항식으로의 변환 원리

1. 테이블을 다선형 확장으로 표현

- 각 테이블의 컬럼은 다선형 확장(MLE)으로 인코딩됩니다
- 행 인덱스는 이진 표현으로 변환되어 다항식의 변수가 됩니다

2. SQL 연산의 다항식 표현

- **SELECT**: 컬럼 선택은 해당 MLE의 평가로 표현
- **WHERE**: 필터링은 Boolean 마스크 다항식으로 표현
- **JOIN**: 테이블 결합은 다항식의 곱셈으로 표현
- **GROUP BY**: 그룹화는 indicator 다항식들의 합으로 표현

3. Sumcheck를 통한 검증

- 모든 SQL 연산이 다항식으로 표현되면, sumcheck 프로토콜을 사용하여 계산의 정확성을 증명할 수 있습니다
- 검증자는 전체 테이블을 보지 않고도 쿼리 결과의 정확성을 확인할 수 있습니다

이 시스템의 핵심은 SQL의 관계형 연산을 다항식 연산으로 변환하고, 영지식 증명 기법을 사용하여 계산의 정확성을 보장하는 것입니다.